Ryerson University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed and Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	w2026

Instructor:	Dr. Muhammad Jaseemuddin

Assignment/Lab Number:	1
Assignment/Lab Title:	Concurrency Vs. Parallelism

Submission Date:	Feb 6, 2026
Due Date:	Feb 6, 2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Pathirana	Chanuth	501107354	01	C.P

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

Introduction

The objective of this lab is to create two programs in Python using threading and multiprocessing modules to process data of several Land Mine Detector Robots. The lab is divided into two main parts, each focusing on a different aspect of data handling and parallel processing. In Part 1, a Python program is created to read a 2D map of a minefield from a file and retrieve movement commands for ten autonomous rovers via a public API. The program then simulates the path of these rovers as they navigate the terrain. In Part 2, the simulation by enabling rover #1 to detect and disarm mines upon receiving a digging command. Using data from a separate file, the disarming process can be simulated through a hashing mechanism.

Implementation

Part 1: Drawing the path of the Rovers

The program first reads map1.txt, where mines are marked as 1 and safe zones as 0, then fetches movement commands for 10 rovers from an API. Each rover has to start at (0,0) and follow its commands to move forward (F), turn (L or R), or dig mines (D); these commands had to be obtained from the public API provided in the lab manual. I also made it so that Pos class keeps track of the current position on the array. Following the lab manual, I made it so that the path for each respective robot is recorded in path_i.txt, with i representing the rover's ID.

```
import threading
import requests
import time
import copy

# ========================
# MAP READING
# ========================

def read_map(filename):
    with open(filename, "r") as f:
        first_line = f.readline().strip().split()
        rows, cols = int(first_line[0]), int(first_line[1])
        land = []
        for _ in range(rows):
            land.append(f.readline().strip().split())
    return rows, cols, land


ROWS, COLS, BASE_MAP = read_map("map1.txt")

# ========================
```

```
# DIRECTION HANDLING
# ========================

DIRECTIONS = ["N", "E", "S", "W"]

MOVE = {
    "N": (-1, 0),
    "E": (0, 1),
    "S": (1, 0),
    "W": (0, -1)
}

def turn_left(d):
    return DIRECTIONS[(DIRECTIONS.index(d) - 1) % 4]

def turn_right(d):
    return DIRECTIONS[(DIRECTIONS.index(d) + 1) % 4]

# ========================
# ROVER EXECUTION LOGIC
# ========================

def run_rover(rover_id):
    land = copy.deepcopy(BASE_MAP)

    x, y = 0, 0
    direction = "S"
    alive = True

    path = [["0" for _ in range(COLS)] for _ in range(ROWS)]
    path[x][y] = "*"

    url = f"http://127.0.0.1:8000/lab1/rover/{rover_id}"  # Change URL to local
FastAPI server
    # print(f"Fetching commands for Rover {rover_id}...")  # Debug line

    # Make the GET request and handle potential key mismatches
    resp = requests.get(url)
    payload = resp.json()

    if resp.status_code != 200:
        print(f"Error: Rover {rover_id} received HTTP {resp.status_code}")
        return

    # Check for different possible response formats
    if isinstance(payload, list):
        commands = payload
    elif "moves" in payload:
        commands = payload["moves"]
```

```
    elif "commands" in payload:
        commands = payload["commands"]
    elif "data" in payload and "moves" in payload["data"]:
        commands = payload["data"]["moves"]
    else:
        raise ValueError(f"Unknown rover response format: {payload}")

    last_command = None

    for cmd in commands:
        if not alive:
            break

        if cmd == "L":
            direction = turn_left(direction)

        elif cmd == "R":
            direction = turn_right(direction)

        elif cmd == "D":
            # Dig current cell
            if land[x][y] == "1":
                land[x][y] = "0"

        elif cmd == "M":
            dx, dy = MOVE[direction]
            nx, ny = x + dx, y + dy

            # Boundary check
            if nx < 0 or nx >= ROWS or ny < 0 or ny >= COLS:
                continue

            # Mine check
            if land[nx][ny] == "1":
                if last_command == "D":
                    land[nx][ny] = "0"
                else:
                    alive = False
                    break

            x, y = nx, ny
            path[x][y] = "*"

        last_command = cmd

    # Write path file
    with open(f"path_{rover_id}.txt", "w") as f:
        for row in path:
            f.write(" ".join(row) + "\n")
```

```python
# ========================
# PART 1 — SEQUENTIAL
# ========================

def run_sequential():
    start = time.time()
    for rover_id in range(1, 11):
        run_rover(rover_id)
    end = time.time()
    return end - start

# ========================
# PART 1 — THREADED
# ========================

def run_threaded():
    threads = []
    start = time.time()

    for rover_id in range(1, 11):
        t = threading.Thread(target=run_rover, args=(rover_id,))
        t.start()
        threads.append(t)

    for t in threads:
        t.join()

    end = time.time()
    return end - start

# ========================
# MAIN
# ========================

if __name__ == "__main__":
    seq_time = run_sequential()
    print(f"Sequential Time: {seq_time:.4f} seconds")

    thread_time = run_threaded()
    print(f"Threaded Time:   {thread_time:.4f} seconds")

    print(f"Time Difference: {seq_time - thread_time:.4f} seconds")
```

Part 2: Digging mines

The code is similar to part 1, except a new function was added called hash, to hash the Temporary Mine Key using the sha256 hash function, as per the lab manual. This is useful for digging and disarming any mine it comes across upon receiving the “D” command.

```
import hashlib
import threading
import time

# ----------------------------
# Load mine serial numbers
# ----------------------------
def load_mines(filename="mines.txt"):
    mines = []
    with open(filename, "r") as f:
        for line in f:
            serial = line.strip()
            if serial:
                mines.append(serial)
    return mines


# ----------------------------
# Brute-force disarm function
# ----------------------------
def disarm_mine(serial):
    pin = 0
    while True:
        temp_key = str(pin) + serial
        h = hashlib.sha256(temp_key.encode()).hexdigest()

        # Check for at least six leading zeros
        if h.startswith("000000"):
            print(f"Disarmed mine with PIN {pin}")
            return pin, h

        pin += 1


# ----------------------------
# Sequential version
# ----------------------------
def sequential_disarm(mines):
    print("\n--- Sequential Disarming ---")
    results = []
    start = time.time()

    for serial in mines:
        result = disarm_mine(serial)
        results.append(result)
```

```
    end = time.time()
    total_time = end - start

    print(f"\nSequential total time: {total_time:.2f} seconds")
    return total_time, results


# ------------------------------
# Thread worker
# ------------------------------
def thread_worker(serial, results, index):
    results[index] = disarm_mine(serial)


# ------------------------------
# Threaded version
# ------------------------------
def threaded_disarm(mines):
    print("\n--- Threaded Disarming ---")
    threads = []
    results = [None] * len(mines)

    start = time.time()

    for i, serial in enumerate(mines):
        t = threading.Thread(target=thread_worker, args=(serial, results, i))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    end = time.time()
    total_time = end - start

    print(f"\nThreaded total time: {total_time:.2f} seconds")
    return total_time, results


# ------------------------------
# Main
# ------------------------------
if __name__ == "__main__":
    mines = load_mines("mines.txt")

    # Sequential
    seq_time, seq_results = sequential_disarm(mines)
```

```python
    # Threaded
    th_time, th_results = threaded_disarm(mines)

    # Comparison
    print("\n--- Comparison ---")
    print(f"Sequential Time: {seq_time:.2f} seconds")
    print(f"Threaded Time:   {th_time:.2f} seconds")
    print(f"Time Difference: {seq_time - th_time:.2f} seconds")
```

Results

Part 1

```
bob@comlab:~/rover-api$ python3 part1.py
Sequential Time: 0.0414 seconds
Threaded Time:   0.0225 seconds
Time Difference: 0.0190 seconds
```

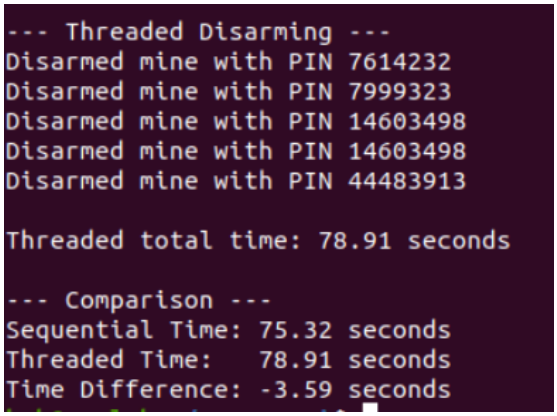
Parallel is faster than sequential by around 0.02 seconds. This makes sense since in the parallel version, there are multiple rovers using threads at the same. In contrast, sequential can only handle rovers one at a time.

Part 2

```
bob@comlab:~/rover-api$ python3 part2.py

--- Sequential Disarming ---
Disarmed mine with PIN 7614232
Disarmed mine with PIN 14603498
Disarmed mine with PIN 14603498
Disarmed mine with PIN 44483913
Disarmed mine with PIN 7999323

Sequential total time: 75.32 seconds
```



Sequential is faster than parallel by around 3.6 seconds. The threaded version is not faster because Python threads do not provide true parallelism for CPU-bound tasks due to the Global Interpreter Lock (GIL). As a result, the threaded version performs similarly or slightly worse than the sequential version due to thread management overhead.

Conclusion

Ultimately, this lab demonstrates efficient handling of concurrent tasks, such as fetching data from multiple API endpoints and processing large datasets in parallel. The concepts explored here are fundamental to optimizing performance in real-world applications involving autonomous robots and sensor networks.